Image Reconstruction with One-Step Drifting Models

Aviram Ohayon (ID 315997866) **Ziv Moshe** (ID 315527176)

Advanced Topics in Signal and Image Processing — Final Project

Abstract

Image-Adaptive GAN (IAGAN) improves generative priors for inverse problems by fine-tuning the generator's weights on each test measurement, which reduces the generator's representation error. In this work we wish to check whether this idea carries over to drifting models, recent one-step generators which make such per-image optimization cheap. The pixel-space drifting model we use takes image-sized Gaussian noise as its input and not a low-dimensional latent. Our method, IADrift, first inverts the input of this pretrained model (CSGM) and then adapts its weights. We fixed the comparison before testing and ran it on noiseless $\times 4$ super-resolution of 100 held-out ImageNet-V2 images with back-projection. With early stopping, adaptation trades a little distortion for a little perceptual quality: compared to CSGM it loses 0.12 dB PSNR and gains 0.008 LPIPS, both significant. Without early stopping it loses 2.92 dB. Our development diagnostics on four images show that the frozen generator has a large representation error on these photos, partly because its discrete style labels are drawn at random. Adapting the weights removes that error when the high-resolution image is fitted directly, but under super-resolution the extra freedom goes mostly into high frequencies, which the measurements do not constrain. As an approximation of the posterior mean, we also average five CSGM reconstructions (added to the protocol after the IADrift test). This improves PSNR over CSGM by 1.57 dB with the pre-declared LR-weighted average and by 2.13 dB with a plain average (secondary). On six development images, the averaging gain exceeds the independent-sample prediction $2/(1 + 1/K)$ for members run for 500 or 1,000 steps and falls below it for 2,000 or 5,000 steps. Also, twelve 500-step members match the PSNR of twelve 5,000-step members at a tenth of the cost. Overall, bicubic upsampling with back-projection keeps the highest PSNR.

1 Introduction

Let us consider the linear inverse problem $\mathbf{y} = \mathbf{A}\mathbf{x}^\star + \mathbf{e}$, where we wish to recover an image $\mathbf{x}^\star$ from its degraded measurements $\mathbf{y}$, e.g. in super-resolution, where $\mathbf{A}$ downsamples the image. When $\mathbf{A}$ has fewer rows than columns, many images explain $\mathbf{y}$ equally well, therefore a prior is needed to choose among them. A popular choice is a learned generative prior: CSGM (Bora et al.) searches the latent space of a pretrained generator for the image that fits the measurements best. Its main weakness is the generator's *representation error*, i.e. the distance between the true image and the closest image the generator can produce. Image-Adaptive GAN (IAGAN) (Abu Hussein et al., 2020) reduces this error by fine-tuning the generator's weights on each test measurement, starting from the CSGM solution.

Modern generative priors, such as diffusion and flow models, have largely displaced GANs. However, they generate by an iterative chain of network evaluations (Deng et al., 2026), so a solver built on them must run or differentiate through many network evaluations. Drifting models (Deng et al., 2026) are recent one-step generators: training moves the generated distribution towards the data distribution, so at inference a sample takes a single forward pass. The idea behind this project is that such a generator brings back the single forward pass that made the test-time optimization of CSGM and IAGAN cheap, but with a modern prior. We use this only as motivation: we did not run a diffusion-based solver for comparison. Concurrent work, DriftSR (Zhu et al., 2026), uses drifting to train a supervised super-resolution network (Section 2.3); we instead invert a pretrained drifting generator per image.

There is also a conceptual difference worth studying. A GAN's prior is enforced geometrically: its latent has a much lower dimension than the image ($k \ll n$), so its output cannot leave a low-dimensional set. The pixel-space drifting model we use maps image-sized Gaussian noise to an image, meaning there is no such bottleneck. Also, part of its input, 32 discrete style labels, cannot

be optimized by gradients at all. Therefore, it is not clear in advance whether the IAGAN recipe carries over to this prior.

In this work we port the recipe to a pretrained pixel-space drifting model and test it on noiseless $\times 4$ super-resolution of ImageNet-V2 images. Our contributions are:

1. IADrift, an image-adaptive drifting generator: CSGM on the Gaussian input followed by adaptation of all weights, evaluated with a protocol fixed before testing (Sections 3 and 4.2). After back-projection, early-stopped adaptation trades a little PSNR for a little LPIPS, and without early stopping it clearly hurts.
2. A diagnosis of why adaptation does not improve PSNR here (Section 4.3): CSGM's gap is largely representation error, partly due to the random style labels, but under super-resolution the extra freedom of the weights goes into the null space of $\mathbf{A}$, mostly into high frequencies, which the measurements do not constrain.
3. An ensemble of CSGM reconstructions, motivated as an approximate posterior mean, which improves both PSNR and LPIPS over CSGM on the test set (Sections 3.4 and 4.4).
4. Open, reproducible code (Section 5).

2 The Existing Method

2.1 Problem setup

As we saw in the introduction, we observe an image $\mathbf{x}^\star \in \mathbb{R}^n$ through a linear operator $\mathbf{A}$:

$$\mathbf{y} = \mathbf{A}\mathbf{x}^\star + \mathbf{e}, \qquad \mathbf{A} \in \mathbb{R}^{m \times n},\ m < n. \tag{1}$$

For super-resolution $\times 4$ (SR$\times$4), $\mathbf{A}$ is antialiased bicubic downsampling from $256 \times 256 \times 3$ to $64 \times 64 \times 3$. Therefore $n = 196{,}608$, $m = 12{,}288$ and the null space of $\mathbf{A}$ has a dimension of at least $n - m = 184{,}320$ (exactly, if $\mathbf{A}$ has full row rank). In frequency terms, $\mathbf{y}$ mainly constrains the content of $\mathbf{x}^\star$ below the low-resolution Nyquist limit $f_{\mathrm{LR}} = 1/8$ cycles/pixel (of the HR grid). Notice that the bicubic kernel is not an ideal low-pass filter, therefore this boundary is soft, so content above f_{LR} must come mostly from a prior. Throughout this report $\mathbf{e} = \mathbf{0}$ (noiseless SR).

2.2 CSGM and image-adaptive GANs

CSGM. Given a pretrained generator $G : \mathbb{R}^k \to \mathbb{R}^n$ with $k \ll n$, CSGM (Bora et al., as formulated by Abu Hussein et al., 2020) estimates the image by solving:

$$\hat{\mathbf{z}} = \arg\min_{\mathbf{z} \in \mathbb{R}^k} \|\mathbf{y} - \mathbf{A}G(\mathbf{z})\|_2^2, \qquad \hat{\mathbf{x}}_{\mathrm{CSGM}} = G(\hat{\mathbf{z}}), \tag{2}$$

using gradient descent through G. The accuracy of CSGM is limited by the generator's *representation error* $E_{\mathrm{rep}}(G, \mathbf{x}) = \min_{\mathbf{z}} \|G(\mathbf{z}) - \mathbf{x}\|_2$, i.e. the distance from $\mathbf{x}$ to the closest image G can produce. For a random Gaussian $\mathbf{A}$, the CSGM error is bounded by a multiple of E_{rep}, and in practice E_{rep} dominates: CSGM plateaus even with many measurements (Abu Hussein et al., 2020, Theorem 1 and Fig. 2, restating Bora et al.).

IAGAN. Abu Hussein et al. (2020) make the generator image-adaptive. Starting from $(\boldsymbol{\theta}_0, \hat{\mathbf{z}}_{\mathrm{CSGM}})$, where $\boldsymbol{\theta}_0$ are the pretrained weights, they solve:

$$(\hat{\boldsymbol{\theta}}, \hat{\mathbf{z}}) = \arg\min_{\boldsymbol{\theta}, \mathbf{z}} \|\mathbf{y} - \mathbf{A}G_{\boldsymbol{\theta}}(\mathbf{z})\|_2^2, \qquad \hat{\mathbf{x}}_{\mathrm{IA}} = G_{\hat{\boldsymbol{\theta}}}(\hat{\mathbf{z}}), \tag{3}$$

with a small learning rate and early stopping. The logic is that the offline semantic prior is kept while the specific image becomes representable. Their motivation is that freeing the weights enlarges the set of images the generator can produce, so E_{rep} can only go down. They prove it for the first layer in the random-Gaussian setting of their Theorem 1 (Abu Hussein et al., 2020, Prop. 2). Notice that neither result covers our bicubic $\mathbf{A}$, therefore we use them only as motivation.

Back-projection (BP). For low noise, Abu Hussein et al. (2020) also enforce *hard* consistency with $\mathbf{y}$ by projecting any estimate $\hat{\mathbf{x}}$ onto $\{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{y}\}$:

$$\mathrm{BP}(\hat{\mathbf{x}}) = \arg\min_{\mathbf{x}} \|\mathbf{x} - \hat{\mathbf{x}}\|_2 \ \text{ s.t. } \ \mathbf{A}\mathbf{x} = \mathbf{y} \ = \ \hat{\mathbf{x}} + \mathbf{A}^\dagger(\mathbf{y} - \mathbf{A}\hat{\mathbf{x}}), \qquad \mathbf{A}^\dagger = \mathbf{A}^\top(\mathbf{A}\mathbf{A}^\top)^{-1}. \quad (4)$$

Let us denote $P_{\mathbf{A}} = \mathbf{A}^\dagger\mathbf{A}$ and $Q_{\mathbf{A}} = \mathbf{I} - P_{\mathbf{A}}$. Then $\mathrm{BP}(\hat{\mathbf{x}}) = P_{\mathbf{A}}\mathbf{x}^\star + Q_{\mathbf{A}}\hat{\mathbf{x}} + \mathbf{A}^\dagger\mathbf{e}$, meaning BP removes the error in the row space of $\mathbf{A}$ and leaves the null-space component of $\hat{\mathbf{x}}$ untouched. As Abu Hussein et al. (2020) suggest for large problems, we solve the $m \times m$ system using conjugate gradients.

2.3 Drifting models

Deng et al. (2026) train a one-step generator $f_{\boldsymbol{\theta}}$ so that its pushforward $q = f_{\boldsymbol{\theta}\#}p_{\boldsymbol{\varepsilon}}$ matches p_{data}. During training, each sample is moved along a *drifting field* $\mathbf{V}_{p,q}$, which is anti-symmetric ($\mathbf{V}_{p,q} = -\mathbf{V}_{q,p}$, hence zero when $q = p$). This is done using the fixed-point loss:

$$\mathcal{L}_{\mathrm{drift}} = \mathbb{E}_{\boldsymbol{\varepsilon}} \sum_j \Big\| \phi_j(\mathbf{x}) - \mathrm{sg}\big(\phi_j(\mathbf{x}) + \mathbf{V}_{p,q}(\phi_j(\mathbf{x}))\big) \Big\|_2^2, \qquad \mathbf{x} = f_{\boldsymbol{\theta}}(\boldsymbol{\varepsilon}), \qquad \mathbf{V}_{p,q} = \mathbf{V}_p^+ - \mathbf{V}_q^-, \quad (5)$$

Where ϕ_j are the features of a pretrained encoder at scale or location j, sg is the stop-gradient, and $\mathbf{V}^\pm$ are kernel mean-shift vectors towards the features of data (positive) and generated (negative) samples (Deng et al., 2026, Eqs. 8, 10 and 14). Generation needs a single forward pass (1 NFE).

The prior we use. We use the released pixel-space model `pixel_B_sota` (a DiT-style B/16 Transformer with 134M parameters; Deng et al., 2026, Table 6 and App. A.2). Its output is the image itself, so $\mathbf{A}$ applies directly:

$$\mathbf{x} = G_{\boldsymbol{\theta}}(\boldsymbol{\varepsilon}, \mathbf{s};\, c), \qquad \boldsymbol{\varepsilon} \in \mathbb{R}^{256\times256\times3}\ (\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})), \quad \mathbf{s} \in \{0, \dots, 63\}^{32}, \quad c \in \{0, \dots, 999\}. \quad (6)$$

The Gaussian input $\boldsymbol{\varepsilon}$ has the dimension of the image and is cut into 16×16 patches, one token each (256 tokens). The 32 *style labels*[1] $\mathbf{s}$ index a learned codebook of 64 embeddings. They are summed into the conditioning vector together with the class c and the guidance scale α, which conditions the network through adaLN and through 16 in-context tokens (Deng et al., 2026, App. A.2). We fix $\alpha = 1$, for which the generator targets the class-conditional data distribution, i.e. no guidance (Deng et al., 2026, Eq. 16), and use the true class c unless stated otherwise.

Concurrent work: drifting for super-resolution. DriftSR (Zhu et al., 2026) uses drifting in a different way. It trains a one-step SR network with supervision, adding a drifting loss to the reconstruction loss and measuring it in the features of a frozen pretrained diffusion model. It does not invert a pretrained generator or adapt weights at test time, therefore its numbers are not comparable with ours. However, it does meet three problems that we also meet:

1. Drifting needs several samples per condition, but SR has one target per input.
2. The prior invents detail that is plausible but inconsistent with the input.
3. Periodic artifacts, to which they suggest the patch tokenization of their prior may contribute.

We return to each of them where it arises (Sections 4.2, 4.3, 4.4, 4.5 and 5).

2.4 Advantages and limitations

IAGAN. *Advantages.* It lowers the representation error without retraining the generator, and it keeps the generator's semantic prior by starting from the CSGM solution and moving the weights only a little. It needs nothing but gradients through G, therefore it applies to any differentiable

[1] Called *random style embeddings* by Deng et al. (2026).

$\mathbf{A}$, and it can be combined with BP. Also, it is cheap on top of CSGM: Abu Hussein et al. (2020) report about 17% extra run time in their compressed-sensing example. *Limitations.* It adds two hyperparameters, the weight learning rate and the stopping step, and the measurements cannot choose them: the measurement loss keeps decreasing while the reconstruction may already be getting worse. When m is small, the extra freedom can overfit the null space of $\mathbf{A}$, hence the small learning rate and early stopping (Section 2.2). Lastly, every step back-propagates through all weights, and each image needs its own copy of them.

The drifting prior. *Advantages.* A sample takes one forward pass, so each CSGM or IAGAN iteration costs one forward and one backward pass, whereas a solver built on a diffusion or flow model must run or differentiate through a whole chain of evaluations (Section 1). Also, training needs no adversarial game, and the sample quality is close to that of multi-step models (FID 1.76 for the pixel-space B/16 model on ImageNet 256×256; Deng et al., 2026, Table 6). This FID uses a guidance scale chosen by a search over $[1, 3.5]$ at inference, and the chosen value is not reported for this model (Deng et al., 2026, Table 8). For their latent models the best FID is at or near $\alpha = 1$ (1.0 for L/2, 1.1 for B/2; Deng et al., 2026, App. B.3), so the number is likely, but not certainly, representative of the unguided prior we use. *Limitations.* The pixel-space model has no low-dimensional latent: $\boldsymbol{\varepsilon}$ has the dimension of the image, therefore there is no geometric bottleneck like a GAN's $k \ll n$. The prior exists only as the pushforward of a Gaussian, so an optimized $\boldsymbol{\varepsilon}$ which leaves the Gaussian shell comes with no guarantee of a natural image. The style labels $\mathbf{s}$ are discrete and cannot be optimized using gradients. The 16×16 patch grid can leave visible tile seams. Finally, the prior is learned by a batch-level objective (5) which per-image inversion does not use (Section 5).

3 Extensions

In this work we pair a drifting prior with image-adaptive inversion, which differs from inverting a GAN in the two ways of Section 1: an image-sized Gaussian input and discrete style labels. The proposal named two extensions, which are stages 1 and 2 below: CSGM on the Gaussian input (Section 3.1) and adaptation of the weights, IADrift (Section 3.2). Two more came out of development: a search over the discrete labels (Section 3.3), and, following the diagnosis in Section 4.3, averaging several stage-1 solutions as an approximate posterior mean (Sections 3.4–3.5). A summary of the methods is presented in Figure 1.

Shared objective. All our methods minimize the same measurement loss and differ only in which variables are free:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\varepsilon}; \mathbf{s}) \;=\; \big\|\mathbf{y} - \mathbf{A}\, G_{\boldsymbol{\theta}}(\boldsymbol{\varepsilon}, \mathbf{s};\, c)\big\|_2^2. \tag{7}$$

For convenience's sake, let us denote by $\mathrm{Adam}_{\eta}^{J}\left(F;\, u^{(0)}\right)$ the iterate after J Adam steps with learning rate η on a function F, started from $u^{(0)}$. No step uses $\mathbf{x}^{\star}$: the HR image is used only for scoring.

3.1 Stage 1: CSGM on the Gaussian input

The weights are frozen at $\boldsymbol{\theta}_0$. The labels $\mathbf{s}$ are discrete, therefore CSGM optimizes only $\boldsymbol{\varepsilon}$. For restarts $r \in \{0, 1\}$ we draw $\boldsymbol{\varepsilon}_r^{(0)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and $\mathbf{s}_r \sim \mathrm{Unif}(\{0, \ldots, 63\}^{32})$, and the CSGM update is:

$$\hat{\boldsymbol{\varepsilon}}_r = \mathrm{Adam}_{0.02}^{5000}\left(\mathcal{L}(\boldsymbol{\theta}_0, \cdot\,; \mathbf{s}_r);\; \boldsymbol{\varepsilon}_r^{(0)}\right) \;\approx\; \arg\min_{\boldsymbol{\varepsilon}}\; \mathcal{L}(\boldsymbol{\theta}_0, \boldsymbol{\varepsilon}; \mathbf{s}_r). \tag{8}$$

Then we keep the restart that fits the measurement best:

$$r^{\star} = \arg\min_{r}\; \mathcal{L}(\boldsymbol{\theta}_0, \hat{\boldsymbol{\varepsilon}}_r; \mathbf{s}_r), \qquad \hat{\mathbf{x}}_{\mathrm{CSGM}} = G_{\boldsymbol{\theta}_0}(\hat{\boldsymbol{\varepsilon}}_{r^{\star}}, \mathbf{s}_{r^{\star}};\, c). \tag{9}$$

As in the CSGM objective of Abu Hussein et al. (2020), we use no regularizer on the input. We chose the learning rate and the absence of a regularizer using a coarse grid search on 100 development images (one per class). The search space was Adam learning rate $\in \{0.005, 0.02\} \times$ no regularizer or a moment penalty on $\boldsymbol{\varepsilon}$ with weight $\in \{1, 10, 100\}$, each run for 3,000 steps with two restarts and ranked by mean LR loss. In the sweep, learning rate 0.02 without a regularizer had both the lowest LR loss and the highest PSNR, and the penalty never helped. We then raised the step count to 5,000 before any test run.

3.2 Stage 2: IADrift, image-adaptive drifting generator

Following (3), we free the weights as well, starting from the CSGM solution and keeping its style labels $\mathbf{s}^\star := \mathbf{s}_{r^\star}$:

$$(\hat{\boldsymbol{\theta}}, \hat{\boldsymbol{\varepsilon}}) \in \arg\min_{\boldsymbol{\theta}, \boldsymbol{\varepsilon}} \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\varepsilon}; \mathbf{s}^\star), \qquad (\boldsymbol{\theta}^{(0)}, \boldsymbol{\varepsilon}^{(0)}) = (\boldsymbol{\theta}_0, \hat{\boldsymbol{\varepsilon}}_{r^\star}). \tag{10}$$

We use two Adam optimizers with separate learning rates, one on all 134M weights and one on the input:

$$\begin{aligned} \boldsymbol{\theta}^{(t+1)} &= \boldsymbol{\theta}^{(t)} - \eta_\theta \ \mathrm{Adam}\big(\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}^{(t)}, \boldsymbol{\varepsilon}^{(t)}; \mathbf{s}^\star)\big), \\ \boldsymbol{\varepsilon}^{(t+1)} &= \boldsymbol{\varepsilon}^{(t)} - \eta_\varepsilon \ \mathrm{Adam}\big(\nabla_{\boldsymbol{\varepsilon}} \mathcal{L}(\boldsymbol{\theta}^{(t)}, \boldsymbol{\varepsilon}^{(t)}; \mathbf{s}^\star)\big), \end{aligned} \tag{11}$$

Where $\mathrm{Adam}(\cdot)$ denotes Adam's bias-corrected moment ratio with default hyperparameters, $\eta_\theta = 10^{-5}$ and $\eta_\varepsilon = 10^{-3}$. This was the setting of our first IADrift runs, and we kept it after a grid search on four development images, $\eta_\theta \in \{10^{-6}, 3 \cdot 10^{-6}, 10^{-5}\} \times \eta_\varepsilon \in \{0, 10^{-3}\}$ (1,000 steps). In this grid, $\eta_\theta = 10^{-5}$ gave the largest gain over CSGM ($+0.30$ dB, at step 50), smaller rates only slowed the later decline, and η_ε made almost no difference. We declared two outputs of the same run before the test run:

$$\hat{\mathbf{x}}_{\mathrm{IADrift@}T} = G_{\boldsymbol{\theta}^{(T)}}(\boldsymbol{\varepsilon}^{(T)}, \mathbf{s}^\star;\, c), \qquad T \in \{100,\ 1000\}. \tag{12}$$

$T = 1000$ is IADrift without early stopping, and $T = 100$ is a fixed early stop. The reason: on the development images PSNR peaked near step 50, both in the joint grid and in the $\boldsymbol{\theta}$-only traces of Section 4.3, which show a mean gain of $+0.28$ dB at step 50 (the peak is at step 50 on three of the four images and at step 100 on the fourth). Choosing the peak step requires the HR image, therefore we declared the round step 100 instead.

Diagnostic variants (development only, Section 4.3): $\boldsymbol{\theta}$*-only* adaptation sets $\eta_\varepsilon = 0$ in (11). *HR fitting* replaces $(\mathbf{A}, \mathbf{y})$ by $(\mathbf{I}, \mathbf{x}^\star)$ in (7), for both CSGM ("CSGM-HR") and IADrift ("IADrift-HR"). Now every frequency is constrained, which isolates representation error from ill-posedness. Notice that it is not a reconstruction method, since it needs the HR image: it only measures how well the generator can represent a given photo.

3.3 Searching the style labels

The full stage-1 problem is mixed discrete–continuous:

$$\min_{\mathbf{s} \in \mathcal{S}} \ \min_{\boldsymbol{\varepsilon}} \ \mathcal{L}(\boldsymbol{\theta}_0, \boldsymbol{\varepsilon}; \mathbf{s}), \qquad |\mathcal{S}| = 64^{32} = 2^{192}, \tag{13}$$

and (8) solves it only for one random $\mathbf{s}$ per restart. We approximate the outer minimum by search:

1. **Candidates:** $\mathcal{C} = \{(r, j) : r \in \{0, 1\},\ j \in \{0, \ldots, 7\}\}$, i.e. the two CSGM starts $\boldsymbol{\varepsilon}_r^{(0)}$, each paired with 8 label draws $\mathbf{s}_{r,j}$, where $\mathbf{s}_{r,0} = \mathbf{s}_r$ is CSGM's own draw.
2. **Screen:** $\ell_{rj} = \mathcal{L}\big(\boldsymbol{\theta}_0,\ \mathrm{Adam}_{0.02}^{500}(\mathcal{L}(\boldsymbol{\theta}_0, \cdot\,; \mathbf{s}_{r,j}); \boldsymbol{\varepsilon}_r^{(0)});\ \mathbf{s}_{r,j}\big)$; keep the two candidates with the smallest ℓ_{rj}, $\mathcal{C}_2 \subset \mathcal{C}$.
3. **Finish:** rerun both finalists for the full 5,000 steps, $\hat{\boldsymbol{\varepsilon}}_{rj} = \mathrm{Adam}_{0.02}^{5000}(\cdot)$, and return $(\hat{r}, \hat{\jmath}) = \arg\min_{(r,j) \in \mathcal{C}_2} \mathcal{L}(\boldsymbol{\theta}_0, \hat{\boldsymbol{\varepsilon}}_{rj}; \mathbf{s}_{r,j})$.

Overall, the cost is $16 \times 500 + 2 \times 5{,}000 = 18{,}000$ steps per image, $1.8\times$ CSGM's 10,000.

3.4 Posterior-mean ensemble

Let us model the truth and each reconstruction as independent posterior samples, $\mathbf{x}^\star = \boldsymbol{\mu} + \mathbf{u}^\star$ and $\mathbf{x}_k = \boldsymbol{\mu} + \mathbf{u}_k$ with $\boldsymbol{\mu} = \mathbb{E}[\mathbf{x} \mid \mathbf{y}]$, where $\mathbf{u}^\star, \mathbf{u}_1, \dots, \mathbf{u}_K$ are i.i.d. with zero mean and covariance Σ. Then:

$$\mathbb{E}\|\mathbf{x}_k - \mathbf{x}^\star\|^2 = 2\,\mathrm{tr}\,\Sigma, \qquad \mathbb{E}\Big\|\tfrac{1}{K}\textstyle\sum_{k=1}^K \mathbf{x}_k - \mathbf{x}^\star\Big\|^2 = \big(1 + \tfrac{1}{K}\big)\mathrm{tr}\,\Sigma, \tag{14}$$

which is a gain of $10\log_{10}\frac{2}{1+1/K} = 2.22$ dB for $K = 5$. CSGM has no regularizer, so each member is a fit to $\mathbf{y}$ from a random start and not a sample from the posterior; this model only motivates the averaging. Also, real members are not independent (they share $\mathbf{y}$, and four of them, restart 0 and its three shifted runs, share a starting point, see below). If $\mathbb{E}[\mathbf{u}_k^\top \mathbf{u}_l] = \rho\,\mathrm{tr}\,\Sigma$ for $k \neq l$, the ensemble error becomes $\big(1 + \frac{1}{K} + \frac{(K-1)\rho}{K}\big)\mathrm{tr}\,\Sigma$, therefore 2.22 dB is an idealized reference rather than a prediction. Our $K = 5$ members are the two CSGM restarts and three CSGM runs on circularly shifted measurements, each started from the initialization $(\boldsymbol{\varepsilon}_0^{(0)}, \mathbf{s}_0)$ of restart $r = 0$:

$$\mathbf{x}_{\mathbf{d}} = S_{-4\mathbf{d}}\,\mathrm{CSGM}\big(S_{\mathbf{d}}\mathbf{y}\big), \qquad \mathbf{d} \in \{(0,1),(1,0),(1,1)\}, \tag{15}$$

Where $S_{\mathbf{d}}$ shifts by $\mathbf{d}$ pixels. The idea: a shift by 4 HR pixels commutes with $\mathbf{A}$ (up to the border) but moves the scene relative to the generator's 16-pixel patch grid, which makes it a cheap source of diversity. The members are weighted by their measurement fit, which gives the LR-weighted ensemble ENS-W:

$$\hat{\mathbf{x}}_{\text{ENS-W}} = \mathrm{BP}\Big(\sum_{k=1}^{5} w_k \mathbf{x}_k\Big), \qquad w_k = \frac{\exp\big(-(L_k - L_{\min})/\tau\big)}{\sum_j \exp\big(-(L_j - L_{\min})/\tau\big)}, \tag{16}$$

Where $L_k = \|\mathbf{y} - \mathbf{A}\mathbf{x}_k\|_2^2$ and $\tau = \mathrm{std}(L_1, \dots, L_5)$. With equal weights $w_k = 1/5$ we call it ENS-U. To separate the two sources of diversity, we also use two uniform averages of subsets of the same members. The *restart average* uses only the two CSGM restarts ($K = 2$). CSGM already runs both restarts and keeps one of them, so this average costs no generator steps beyond CSGM. The *shift average* uses restart 0 and its three shifted runs ($K = 4$), so its members differ only in their grid phase and not in their starting point.

3.5 Larger ensembles: more style labels, shorter runs

ENS-W and ENS-U have only two independent starts, because the shifted members reuse restart 0's $(\boldsymbol{\varepsilon}_0^{(0)}, \mathbf{s}_0)$. To test (14) as a function of K, we ran a development experiment on six images. It has $N = 12$ CSGM members per image, each from its own draw $(\boldsymbol{\varepsilon}_r^{(0)}, \mathbf{s}_r)$ with $r = 0, \dots, 11$ (members 0 and 1 are the two CSGM restarts). Each member is saved after $T \in \{500, 1000, 2000, 5000\}$ steps; saving does not change the trajectory, so one run gives all four budgets. For each $K \in \{1, 2, 4, 5, 8, 12\}$ and each T we average up to 20 random K-subsets, either uniformly or with the weights of (16), and then apply BP. We compare the measured gain $10\log_{10}\big(\overline{\mathrm{MSE}}_{\mathrm{mem}}/\mathrm{MSE}_{\mathrm{ens}}\big)$ (before BP, where $\overline{\mathrm{MSE}}_{\mathrm{mem}}$ is the members' mean) with the prediction $10\log_{10}\frac{2}{1+1/K}$ of (14). As everywhere in this report, the gain is computed per image and then averaged over images. Two more arms reuse these members:

1. The first runs IADrift@100 from each member's 5,000-step solution.
2. The second crosses 4 label draws with 4 grid phases, the unshifted measurement plus the three shifts of (15) ($K = 16$), to compare the two sources of diversity.

An ensemble costs $K \cdot T$ steps, so $12 \times 500 = 6{,}000$ steps is less than CSGM's 10,000. This experiment was designed after the test runs and was not run on the test set.

Averaging over labels as a mixture. The labels are a discrete latent of the prior, so the posterior splits over them. Given $(\boldsymbol{\varepsilon}, \mathbf{s})$ the generator is deterministic, $p(\mathbf{x} \mid \boldsymbol{\varepsilon}, \mathbf{s}) = \delta\big(\mathbf{x} - G_{\boldsymbol{\theta}_0}(\boldsymbol{\varepsilon}, \mathbf{s};\, c)\big)$, and

therefore:

$$p(\mathbf{x} \mid \mathbf{y}) = \sum_{\mathbf{s} \in \mathcal{S}} p(\mathbf{s} \mid \mathbf{y})\, p(\mathbf{x} \mid \mathbf{y}, \mathbf{s}), \qquad \boldsymbol{\mu} = \sum_{\mathbf{s} \in \mathcal{S}} p(\mathbf{s} \mid \mathbf{y})\, \mathbf{m}(\mathbf{s}),$$
$$\mathbf{m}(\mathbf{s}) = \mathbb{E}[\mathbf{x} \mid \mathbf{y}, \mathbf{s}] = \mathbb{E}_{\boldsymbol{\varepsilon} \sim p(\boldsymbol{\varepsilon} \mid \mathbf{y}, \mathbf{s})} \big[G_{\boldsymbol{\theta}_0}(\boldsymbol{\varepsilon}, \mathbf{s};\, c) \big]. \tag{17}$$

Notice two things. First, the weights are $p(\mathbf{s} \mid \mathbf{y}) \propto p(\mathbf{s})\, p(\mathbf{y} \mid \mathbf{s})$ and not the prior $p(\mathbf{s})$, because $\mathbf{y}$ carries information about the labels: the label search of Section 3.3 lowers the final LR loss from 50.5 to 30.0 (Section 4.5). Second, $\boldsymbol{\varepsilon}$ is integrated out and not conditioned on: given $\boldsymbol{\varepsilon}$ as well, $\mathbf{x}$ is fixed and there is nothing left to average. A uniform average of members with $\mathbf{s}_k \sim \mathrm{Unif}(\mathcal{S})$ is a Monte Carlo estimate of (17) with the prior as weights. The LR weights of (16) act like the likelihood $p(\mathbf{y} \mid \mathbf{s}_k)$ in self-normalized importance sampling, but only loosely. With $\mathbf{e} = \mathbf{0}$, $p(\mathbf{y} \mid \mathbf{s}) = \int p(\boldsymbol{\varepsilon})\, \delta\big(\mathbf{y} - \mathbf{A} G_{\boldsymbol{\theta}_0}(\boldsymbol{\varepsilon}, \mathbf{s};\, c)\big)\, d\boldsymbol{\varepsilon}$ has no closed form, while $\exp(-L_k/\tau)$ uses only the member's own fit and a τ set from the data. Moreover, CSGM fits $\boldsymbol{\varepsilon}$ without its prior, so a member is a point of the fit set reached from a random start, and not $\mathbf{m}(\mathbf{s}_k)$.

Still, the mixture changes the prediction. Suppose each member is exactly $\mathbf{m}(\mathbf{s}_k)$ with $\mathbf{s}_k \sim p(\mathbf{s} \mid \mathbf{y})$ i.i.d. Let $W = \mathbb{E}_{\mathbf{s}} \operatorname{tr} \mathrm{Cov}(\mathbf{x} \mid \mathbf{y}, \mathbf{s})$ be the variance within labels and $B = \operatorname{tr} \mathrm{Cov}_{\mathbf{s}}\, \mathbf{m}(\mathbf{s})$ the variance between them, so that $W + B = \operatorname{tr} \Sigma$ by the law of total variance, and let $\beta = B / \operatorname{tr} \Sigma$. Then:

$$\mathbb{E}\|\mathbf{x}_k - \mathbf{x}^\star\|^2 = (1 + \beta) \operatorname{tr} \Sigma, \qquad \mathbb{E}\Big\| \tfrac{1}{K} \textstyle\sum_{k=1}^{K} \mathbf{x}_k - \mathbf{x}^\star \Big\|^2 = \big(1 + \tfrac{\beta}{K}\big) \operatorname{tr} \Sigma,$$
$$\text{gain} = 10 \log_{10} \frac{1 + \beta}{1 + \beta/K}. \tag{18}$$

For $\beta = 1$ this is (14). For $\beta < 1$, the variance within labels is in the error of every member and of the average alike, so the gain stays below (14) and saturates at $10 \log_{10}(1 + \beta) < 3\,\mathrm{dB}$. Algebraically, (18) is the correlated version of (14) with $\rho = (1 - \beta)/(1 + \beta)$, so the two models fit the measured gains equally well but read the shortfall differently. Our members also differ in their start $\boldsymbol{\varepsilon}^{(0)}$, which covers part of the spread within labels; (18) is the idealized case in which it does not. The shifted members of (15) share $\mathbf{s}_0$ and differ only in grid phase, so this decomposition does not describe them.

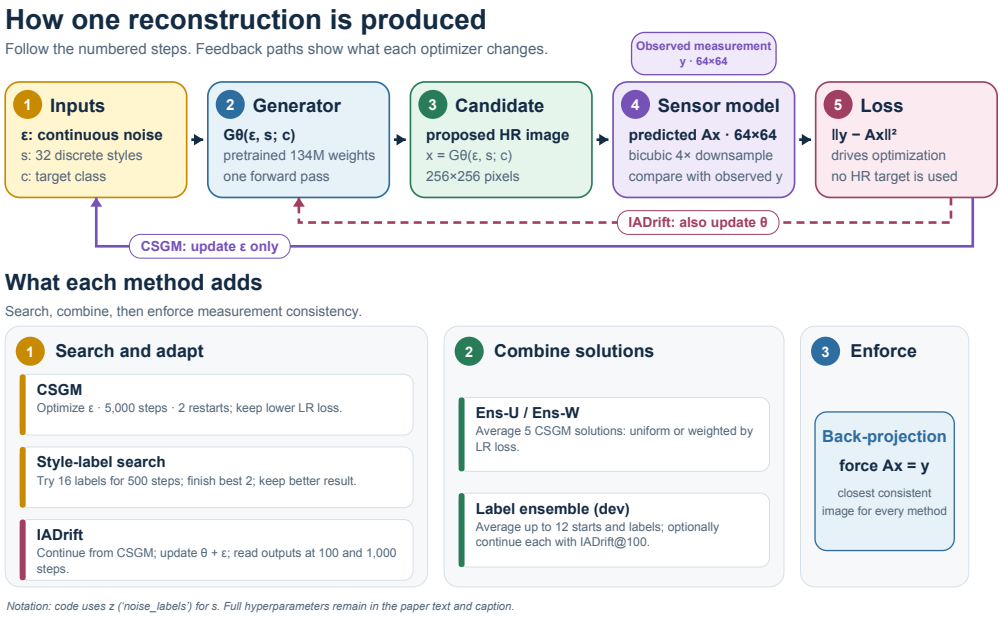


Figure 1: Overview of the methods. *Top:* one reconstruction. The generator maps the Gaussian input $\boldsymbol{\varepsilon}$, the discrete style labels $\mathbf{s}$ and the class c to a 256×256 candidate $\mathbf{x}$. The candidate is downsampled by $\mathbf{A}$ and compared with the observed $\mathbf{y}$, and this LR loss is the only training signal. CSGM updates only $\boldsymbol{\varepsilon}$, with the weights frozen at $\boldsymbol{\theta}_0$; IADrift also updates $\boldsymbol{\theta}$. *Bottom:* each method first searches (CSGM, style-label search) or adapts (IADrift), may average several solutions (ENS-U, ENS-W, label ensemble), and finally back-projects onto $\{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{y}\}$. The label ensemble was run only on the development set.

4 Results

4.1 Experimental setup

Data. The drifting generator was trained on ImageNet, therefore we evaluate on a separate dataset with the same classes: ImageNet-V2 MatchedFrequency, a test set collected independently of ImageNet with the same 1,000 classes and a matched selection procedure. It has 10 images for each class, which we split per class into 8 development and 2 test images (seed 2026). The test set is 100 test images from 100 distinct classes (seed 2028). HR images are resized (bicubic) to a short side of 256, centre-cropped to 256×256 and scaled to $[-1, 1]$. All settings were chosen on development images: the CSGM sweep used 100 (one per class); the diagnostics, the ε projection and the style-label search 4; the image-in-ε test 3; and the ensemble experiments 6. The variance decomposition of Section 4.5 uses generator samples of 8 classes. Test image 1704 was used for smoke tests of CSGM and IADrift before the benchmark (nothing was tuned on it), and it is the first image of the fixed test sample.

Metrics. PSNR uses data range 2 and is averaged over images. LPIPS uses AlexNet features (IAGAN reports the same metric as "perceptual similarity"). To locate errors in frequency we use the band SNR $10 \log_{10} \left(\|P_b \mathbf{x}^\star\|^2 / \|P_b(\hat{\mathbf{x}} - \mathbf{x}^\star)\|^2 \right)$, where P_b keeps the radial frequencies of band b of the Hann-windowed image: low $[0, \frac{1}{16})$, mid $[\frac{1}{16}, \frac{1}{8})$, high $[\frac{1}{8}, \frac{1}{4})$ and very high $[\frac{1}{4}, \infty)$ cycles/pixel. The first two bands lie below f_{LR} and the last two above it. The high band also contains the corners of the square $|f_x|, |f_y| < f_{\mathrm{LR}}$ that $\mathbf{y}$ measures, so it is not entirely unconstrained. An empty band scores 0 dB, meaning a negative value indicates that the invented content is worse than none. The generator works on 16×16 patches and can leave visible seams along the patch borders (tile seams). To measure them in a reconstruction we use a *seam score*: the mean absolute jump between neighbouring pixels across the 16-pixel tile borders, divided by the mean jump at the other 4-pixel-aligned positions. An image without seams scores about 1 (the HR test images score 1.01), and higher values mean more visible seams. The seam score is computed on reconstructions; it is not the grid score of Section 4.5, which we use only on generator samples.

Protocol. We wrote down the test comparison before IADrift ran on any test image. It has four primary tests: IADrift@100 and IADrift@1000 against CSGM, after BP, in PSNR and in LPIPS. Each test is paired over the 100 images and reports the mean difference with a normal-approximation 95% CI, mean $\pm 1.96\, \hat{\sigma}/\sqrt{100}$. Here $\hat{\sigma}$ is the sample standard deviation of the 100 per-image differences, so $\hat{\sigma}/\sqrt{100}$ is the standard error of their mean. By the central limit theorem the mean is approximately normal, and 1.96 is the 97.5th percentile of the standard normal distribution. Each test also reports a Wilcoxon signed-rank p, Holm-corrected over the four tests, and the number of images on which the method is better. The ensembles were added in an addendum after the IADrift test but before their own test run: ENS-W is its primary method (two Holm-corrected tests) and ENS-U a secondary one. The same addendum declared the restart average and the shift average of Section 3.4 as secondary methods, so they were not chosen after seeing the test results. Everything else (results without BP, a wrong class label, all development results) is secondary and uncorrected. Overall, no step selects, weights or stops using the HR image.

Compute. All runs used one RTX A4000, and the timings below are approximate. A 5,000-step CSGM run takes about 1 min (so about 2 min per image with two restarts) and a 1,000-step IADrift run about 1 min. IADrift@100 is not a separate run: it is the step-100 checkpoint of the 1,000-step run, so its share of the cost is about 6 s. The test runs took about 16 GPU-hours and the CSGM sweep about 22.

Deviations from the proposal. Some changes were made with respect to the proposal:

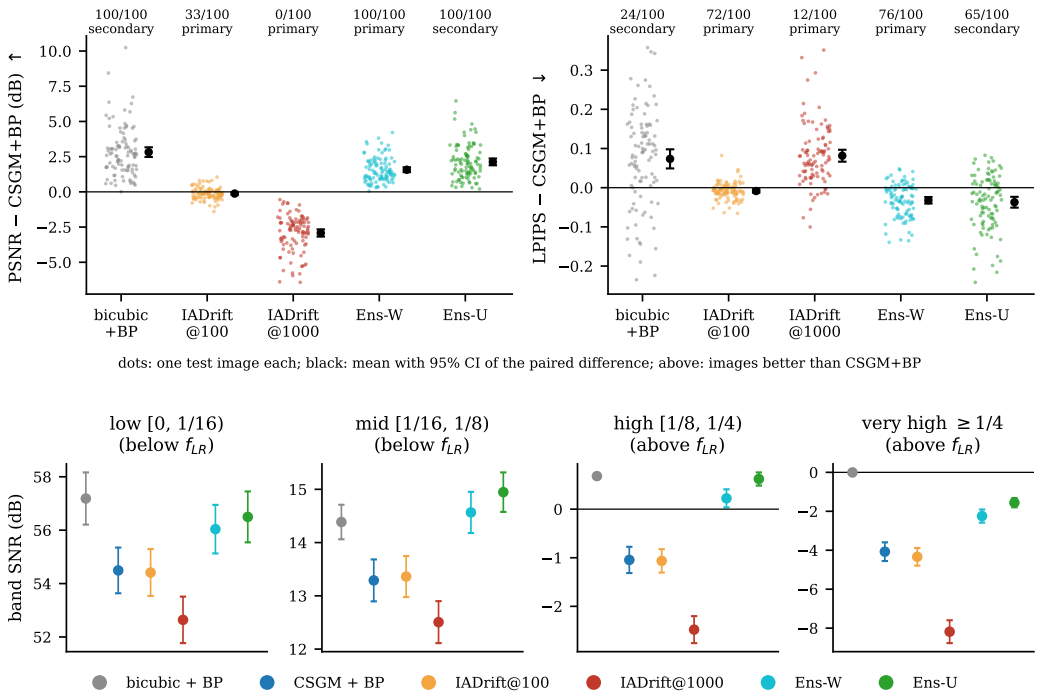


Figure 2: Test set, 100 images, all after BP. *Top:* per-image difference from CSGM + BP in PSNR and LPIPS (dots), with the mean and the 95% CI of the paired difference (black; for the primary tests these are the values of Table 2). Above each method: the number of images on which it beats CSGM + BP. *Bottom:* band SNR (mean and 95% CI over images) in the four bands of Section 4.1; 0 dB is what an empty band scores.

1. We used ImageNet-V2 instead of the ImageNet-1k validation set, for the reason given under *Data*.
2. SR×8 and noisy SR ($\sigma = 10/255$) were not run. The results on noiseless SR×4, the easiest of the planned settings, were unsatisfactory: CSGM stayed well below bicubic in PSNR, and adaptation lowered PSNR further, with only a small LPIPS gain. Rather than move on to harder settings, we first investigated how the generator behaves on this one (Sections 4.3 and 4.5), which led to the ensembles.

4.2 Main result: after back-projection, early-stopped adaptation trades PSNR for LPIPS

Presented in Table 1(a), Table 2 and Figures 2 and 3 are the protocol results. In the examples of Figure 3, CSGM adds sharp but wrong texture, IADrift@1000 adds more of it, and ENS-U removes much of it.

The existing method on this prior. CSGM + BP is 2.82 dB below bicubic + BP in PSNR and lower on all 100 images, but its LPIPS is lower (0.284 against 0.358, better on 76/100). Meaning, the generator adds plausible detail that is not the true detail. This is the usual perception–distortion tradeoff, which Abu Hussein et al. (2020) also observe for IAGAN. Full-reference metrics such as PSNR favour smooth estimates, so a smooth baseline like bicubic tends to lead in PSNR. IAGAN's results are mixed: on CelebA-HQ at ×8 and ×16 bicubic has a higher PSNR than every method, but on CelebA at ×4 and ×8 IAGAN-BP beats it (27.14 against 26.50 dB at ×4) (Abu Hussein et al., 2020, Table 2). So bicubic's PSNR lead is expected for a prior that invents detail, but not guaranteed. BP raises PSNR on every image for every method, but this is guaranteed rather than found: BP is the

Table 1: Every method in the report, with back-projection except the rows marked "no BP" and block (c). Each block uses its own image set and has its own bicubic and CSGM reference, so compare numbers within a block only. Steps are Adam steps through the generator per image (10k = CSGM with two 5,000-step restarts; IADrift-HR runs 2,000 steps from CSGM-HR). Blocks (b)–(d) are development results with no significance tests; block (d) was designed after the test runs. Block (c) fits the HR image itself ($\mathbf{A} = \mathbf{I}$, no BP), so it is not a reconstruction (Section 3.2); compare it with no other block. U/W = uniform/LR-weighted average. Restart average: the mean of CSGM's two restarts; shift average: restart 0 and its three shifted runs (Section 3.4). Differences quoted in the text are computed before rounding. Bold: best in the block (none in (c)).

Method	Steps	PSNR (dB) ↑	LPIPS ↓
(a) Test set, 100 images: the protocol (Sections 4.2 and 4.4)			
Bicubic	–	**25.57**	0.358
CSGM	10k	22.75	0.284
IADrift@100	10.1k	22.63	0.276
IADrift@1000	11k	19.83	0.366
Restart average, 2 members (addendum, secondary)	10k	23.76	0.259
Shift average, 4 members (addendum, secondary)	20k	24.69	0.253
ENS-W (addendum, primary)	25k	24.32	0.252
ENS-U (addendum, secondary)	25k	24.88	**0.247**
(b) Development, 4 images: diagnostics and ablations (Sections 4.3 and 4.5)			
Bicubic	–	**25.43**	0.429
CSGM	10k	21.86	0.328
IADrift@1000	11k	18.31	0.467
IADrift@1000 (no BP)	11k	18.30	0.470
$\boldsymbol{\theta}$-only adaptation, 2,000 steps (no BP)	12k	18.18	0.481
CSGM, $\boldsymbol{\varepsilon}$ projected per patch onto the shell	10k	21.99	0.373
CSGM, $\boldsymbol{\varepsilon}$ projected onto the global sphere	10k	21.72	0.388
Style-label search	18k	22.10	**0.314**
(c) Representation only, not SR: the HR image fitted directly ($\mathbf{A} = \mathbf{I}$), same 4 images			
CSGM-HR, frozen weights	10k	24.44	0.332
IADrift-HR, adapted weights	12k	40.09	0.030
(d) Development, 6 images: ensembles (Section 4.4)			
Bicubic	–	**25.73**	0.418
CSGM	10k	22.45	0.303
Single member, 500 / 5,000 steps (mean of 12)	0.5k / 5k	22.40 / 23.16	0.425 / 0.334
IADrift@100 / @1000	10.1k / 11k	22.14 / 19.05	0.304 / 0.421
ENS-U / ENS-W, 5 members	25k	24.81 / 24.04	0.302 / 0.273
IADrift@100 ensemble, 5 members, U / W	25.5k	24.71 / 23.82	0.293 / 0.273
IADrift@1000 ensemble, 5 members, U	30k	23.00	0.318
12×500 steps, distinct labels, U / W	6k	25.05 / 24.92	0.315 / 0.307
$12 \times 5{,}000$ steps, distinct labels, U / W	60k	25.10 / 24.96	0.246 / 0.238
$12 \times (5{,}000 + \text{IADrift@100})$, U / W	61.2k	24.98 / 24.77	0.238 / **0.227**
4 labels $\times$ 4 grid phases ($K = 16$), U / W	80k	25.47 / 25.28	0.294 / 0.264

Table 2: Primary paired tests against CSGM + BP over the 100 test images: mean difference with a normal-approximation 95% CI, Holm-corrected Wilcoxon p, and the number of images on which the method is better. The IADrift rows are corrected over 4 tests and the ensemble rows over 2.

Method (+ BP)	Metric	Difference [95% CI]	Holm p	Better
IADrift@1000	PSNR (dB)	-2.92 $[-3.18, -2.66]$	1.6×10^{-17}	0/100
	LPIPS	$+0.082$ $[+0.066, +0.097]$	1.0×10^{-14}	12/100
IADrift@100	PSNR (dB)	-0.12 $[-0.20, -0.04]$	1.4×10^{-3}	33/100
	LPIPS	-0.008 $[-0.013, -0.004]$	3.8×10^{-5}	72/100
ENS-W	PSNR (dB)	$+1.57$ $[+1.40, +1.74]$	7.8×10^{-18}	100/100
	LPIPS	-0.032 $[-0.040, -0.024]$	5.7×10^{-10}	76/100

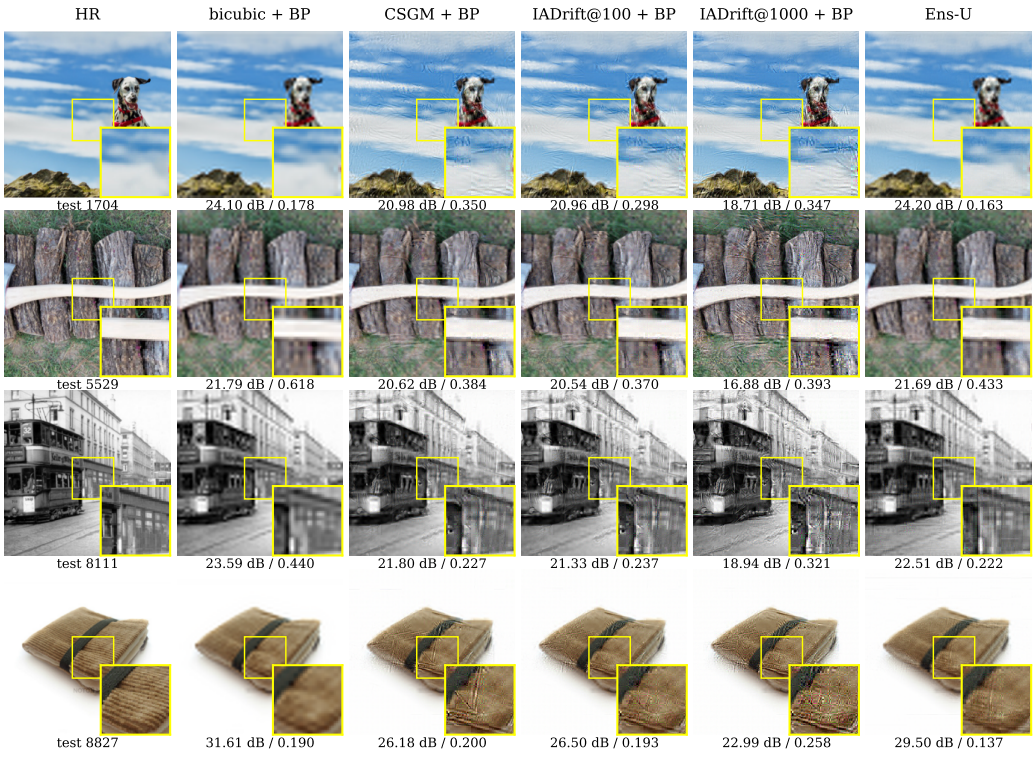


Figure 3: The first four images of the fixed test sample (not chosen by result; image 1704 is the disclosed smoke-test image), after BP. Below each image: PSNR / LPIPS for that image. Inset: the yellow 64×64 crop, which starts at pixel 104 and so lies off the generator's 16-pixel grid.

orthogonal projection onto an affine set that contains $\mathbf{x}^\star$ ($\mathbf{e} = \mathbf{0}$), so it cannot increase the distance to $\mathbf{x}^\star$. Therefore we use it as a sanity check. The LPIPS change, on the other hand, is not guaranteed: BP lowers LPIPS on 100/100 images for every method except IADrift@1000 (92/100). As for the class label, it matters little for PSNR: with a random wrong label, CSGM + BP scores 22.81 dB against 22.75 dB with the correct one ($+0.06$ dB $[-0.22, +0.34]$). It does matter for LPIPS, which is worse with the wrong label (0.302 against 0.284; $+0.018$ $[+0.004, +0.032]$, Wilcoxon $p = 0.03$, uncorrected).

IADrift. Without early stopping, adaptation clearly hurts: IADrift@1000 loses 2.92 dB and is worse on every image. With the declared early stop, IADrift@100 trades distortion for perceptual quality: it loses 0.12 dB (better on 33/100) and gains 0.008 in LPIPS (better on 72/100). Both differences are significant after Holm correction (Holm $p = 1.4 \times 10^{-3}$ and 3.8×10^{-5}), but both are small: the LPIPS gain is about 3% of CSGM's LPIPS. Interestingly, without BP (secondary) IADrift@100 does beat CSGM, by $+0.28$ dB (81/100) and -0.028 LPIPS (95/100). Most of that advantage is a better fit of $\mathbf{y}$ in the row space of $\mathbf{A}$, which BP gives every method for free.

4.3 Why adaptation does not improve PSNR: development diagnostics

Apart from the test-set band SNR (Figure 2, bottom) and the test-set loss curve in Figure 4, these diagnostics use four development images and no significance tests, therefore we read them as exploratory.

Fitting $\mathbf{y}$ better does not give a better $\mathbf{x}$. During adaptation on the test set, the median LR loss drops from 38.5 to 0.98, and on 99 of the 100 images it falls more than tenfold (median 43-fold; smallest 9.9-fold). Meanwhile, PSNR after BP drops from 22.75 to 19.83 dB. Presented in Figure 4a are the development traces, which show the same divergence step by step.

Representation or ill-posedness? To separate the two, we leave SR and fit the HR image itself, replacing $(\mathbf{A}, \mathbf{y})$ with $(\mathbf{I}, \mathbf{x}^\star)$, which constrains every frequency. This uses $\mathbf{x}^\star$, so it shows only how well the ImageNet-trained generator can represent these four development photos (Table 1(c)). The frozen generator then reaches only 24.4 dB (CSGM-HR), below bicubic + BP from $\mathbf{y}$ (25.4 dB; on 3 of the 4 images), meaning its representation error on these photos is large (Figure 4, bottom). CSGM-HR only bounds that error from above: it keeps one random draw of $\mathbf{s}$ per restart and runs a finite number of steps. With its weights adapted, the generator reaches 40.1 dB and LPIPS 0.030 (IADrift-HR), so on these photos weight adaptation can remove the representation error, as IAGAN assumes. This shows capacity only; it does not show that the prior is kept. To separate the parts of CSGM's gap, we ran an in-range experiment. For the class c of each of the four images we generate a target the generator produces exactly, $\mathbf{x}^\star = G_{\boldsymbol{\theta}_0}(\boldsymbol{\varepsilon}^\star, \mathbf{s}^\star; c)$ with random $(\boldsymbol{\varepsilon}^\star, \mathbf{s}^\star)$, and run CSGM from $\mathbf{y}^\star = \mathbf{A}\mathbf{x}^\star$. With the true labels $\mathbf{s}^\star$, CSGM reaches 25.9 dB and LPIPS 0.116 without BP, above bicubic + BP (23.0 dB, 0.402). With random labels, as in our method, it falls to 21.9 dB and 0.331, although $\boldsymbol{\varepsilon}$ is free in both runs. The labels set most of an image's colour and spectrum (Section 4.5), and $\boldsymbol{\varepsilon}$, the only input CSGM can optimize, cannot make up for wrong ones. Therefore, for generated targets of these four classes, most of CSGM's gap comes from the wrong labels. CSGM on the real photos reaches a similar PSNR (21.1 dB without BP, against 21.9 dB in range with random labels), so the same limit probably explains much of its gap there, and much of CSGM-HR's 24.4 dB as well. IADrift-HR keeps the same random draw and still reaches 40.1 dB, meaning the adapted weights can compensate for wrong labels where $\boldsymbol{\varepsilon}$ cannot. To conclude, adaptation could remove that error in principle, but under SR the same adaptation lowers PSNR: the extra freedom goes into what $\mathbf{y}$ leaves unconstrained (next paragraph).

Where the freedom goes. After BP the error is $\mathrm{BP}(\hat{\mathbf{x}}) - \mathbf{x}^\star = Q_{\mathbf{A}}(\hat{\mathbf{x}} - \mathbf{x}^\star)$ by (4), i.e. it lies entirely in the null space. The loss gradient with respect to the image, $-2\mathbf{A}^\top(\mathbf{y} - \mathbf{A}\hat{\mathbf{x}})$, lies in the

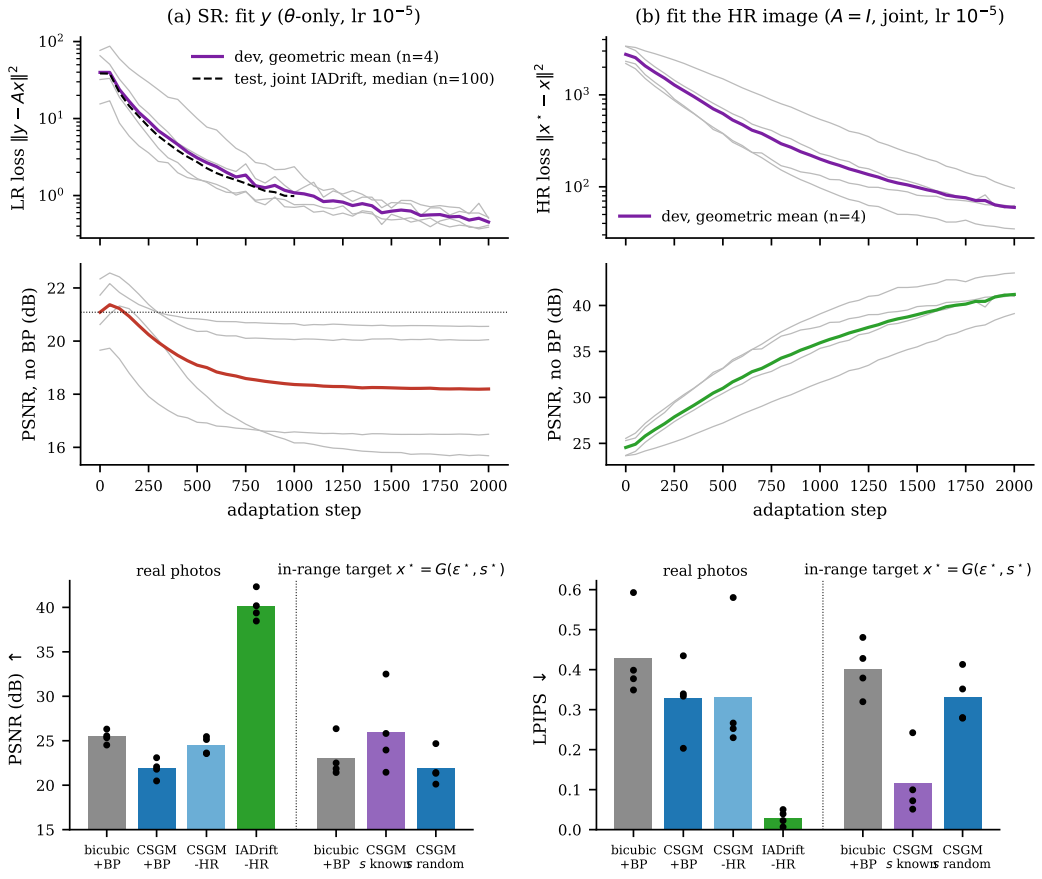


Figure 4: *Top: fitting* $\mathbf{y}$ *better makes* $\mathbf{x}$ *worse* (development, 4 images; grey: single images). (a) Under SR, adapting the weights ($\boldsymbol{\theta}$-only, the setting for which we recorded PSNR at every step) keeps lowering the LR loss while PSNR (no BP) falls after step 50; the dashed line is the median LR loss of joint IADrift on the 100 test images. (b) The same adaptation fitted to the HR image ($\mathbf{A} = \mathbf{I}$) raises PSNR above 40 dB. *Bottom: representation error* (development, 4 images; dots: single images). Left of the dotted line, real photos: SR methods after BP, and the fits to the HR image itself ($\mathbf{A} = \mathbf{I}$, no BP; not reconstructions, Section 3.2). Right of it, in-range SR, with a generated target of the same class that the generator can produce exactly, and CSGM run with the true style labels $\mathbf{s}^\star$ or with random ones (no BP), each scored against its own target.

row space, therefore the loss carries no information about $Q_{\mathbf{A}}\hat{\mathbf{x}}$. That component changes only as a side effect of the generator's Jacobian, and with 134M free weights nothing ties it to $\mathbf{x}^\star$. The band SNR on the 100 test images (Figure 2, bottom) is consistent with this:

1. Bicubic + BP is near 0 dB above f_{LR} ($+0.7$ dB high, 0.0 dB very high; it leaves those bands nearly empty).
2. CSGM + BP is below that (-1.0 / -4.1 dB), and IADrift@1000 lowers it further (-2.5 / -8.2 dB).
3. IADrift@1000 also loses SNR below f_{LR} (-1.9 dB low, -0.8 dB mid against CSGM + BP), but these bands keep an SNR above 12 dB, so the error added there is small: on the four example test images of Figure 3, 99.5% of the error energy that IADrift@1000 adds lies above f_{LR} (81% in the very-high band). IADrift@100 stays within 0.3 dB of CSGM + BP in every band.

The tile seams are not the main source of this error: IADrift@1000 removes most of them (seam score 1.04, against 1.26 for CSGM + BP and 1.01 for the HR images) while its PSNR falls by 2.92 dB. Also, the boundary at f_{LR} is soft: on the development images, bicubic + BP still has an SNR of $+3.7$ dB just above f_{LR} (0.13 cycles/pixel) and reaches 0 dB only near 0.17. DriftSR (Zhu et al., 2026) meets the same problem in supervised training, where drifting "may encourage visually plausible textures that are inconsistent with the input". Their fix also splits by frequency: a second drifting field computed on wavelet low-frequency components, given more weight wherever the output's local gradient structure agrees less with the low-resolution input than the HR image's does. However, that map needs the HR image, which they have during training and we do not have at test time. BP is the only consistency we can enforce, and it fixes the row space only.

The weights cause it. Without BP and at the same step 1,000, adapting only $\boldsymbol{\theta}$ ($\eta_\varepsilon = 0$) gives 18.36 dB and joint IADrift gives 18.30 dB; after 2,000 steps, $\boldsymbol{\theta}$-only reaches 18.18 dB. Therefore, moving $\boldsymbol{\varepsilon}$ plays no visible part in the drop. This agrees with the caution of Abu Hussein et al. (2020) that a small learning rate and early stopping are needed when m is small. Here $m/n = 1/16$, so the null space covers at least $15/16$ of the dimensions, and the generator has 134M weights.

4.4 What helps: averaging reconstructions

Test set. The pre-declared ENS-W improves on CSGM + BP by $+1.57$ dB (better on 100/100) and -0.032 LPIPS (76/100), and both tests are significant (Table 2). The uniform ENS-U (secondary, uncorrected) does better in PSNR: $+2.13$ dB (100/100). Its LPIPS of 0.247 is better than CSGM's on 65/100 images, but it does not differ significantly from ENS-W's (-0.005 $[-0.012, +0.002]$, Wilcoxon $p = 0.51$), so we do not rank the two by LPIPS. ENS-U beats ENS-W in PSNR on 90/100 images, because the LR weights concentrate on a few members and so give up part of the averaging (an effective $1/\sum_k w_k^2$ of 2.9 of the 5 members on average, range 1.7–4.2). The two smaller secondary averages (Section 3.4; uncorrected) show where the gain comes from:

1. The restart average (no generator steps beyond CSGM) already gains $+1.02$ dB $[0.84, 1.19]$ (better on 89/100, Wilcoxon $p = 8 \times 10^{-16}$) and -0.025 LPIPS $[-0.034, -0.015]$ (67/100).
2. The shift average gains $+1.94$ dB $[1.70, 2.18]$ (98/100) and -0.031 LPIPS (63/100).

So most of ENS-U's gain comes from the three shifted runs: adding the second restart to them (ENS-U) adds only $+0.19$ dB $[0.15, 0.24]$. Bicubic + BP keeps the highest PSNR: ENS-U is 0.69 dB below it (better on 11/100) but much better in LPIPS (0.247 against 0.358, better on 92/100). Before BP, the measured gain of ENS-U over its members (the mean of the per-image dB gains, as in Section 3.5) is 2.33 dB $[2.18, 2.48]$, near the 2.22 dB which (14) predicts for $K = 5$; with MSEs pooled over images instead it is 2.14 dB. However, we do not read this as a confirmation of (14): on the development images, the gain of 5,000-step members falls short of the prediction at every K (Figure 5). Also, averaging raises the band SNR in every band (low $+2.0$, mid $+1.7$, high $+1.7$, very high $+2.5$ dB against CSGM + BP; Figure 2), not only above f_{LR}. It is less usual that the average also lowers LPIPS: the posterior mean is the smooth estimate that SR methods usually try to avoid,

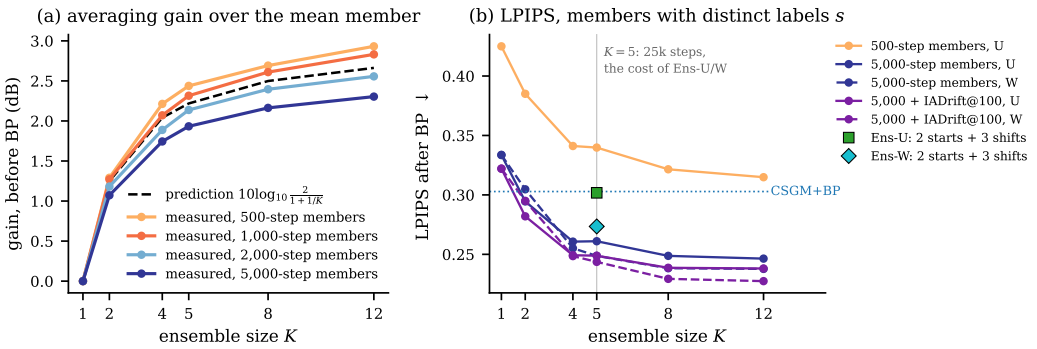


Figure 5: Development (6 images); each member has its own $(\boldsymbol{\varepsilon}, \mathbf{s})$, and each point is the mean over up to 20 random K-subsets of the 12 members. *(a)* The measured gain of a uniform average of K members over the mean member (before BP; mean over images of the per-image dB gain), for each step budget, against the prediction of (14) for independent posterior samples. *(b)* LPIPS after BP of the same ensembles (U uniform, W LR-weighted). The markers at $K = 5$ are ENS-U and ENS-W on the same six images: also five 5,000-step members (25k steps), but two independent starts and three grid shifts of the first.

and DriftSR finds that reconstruction-only training over-smooths (Zhu et al., 2026, Fig. 1). Here the members start from detail that is worse than none above f_{LR} (negative band SNR), so averaging removes more wrong texture than true texture. A prior that invents less would leave less for the average to remove.

Larger ensembles (development, Section 3.5). Figures 5 and 6 and Table 1(d) present the results, on six images and without tests. On all six images bicubic + BP still has the highest PSNR (25.73 dB). In the crops of Figure 6, most of CSGM's false texture is gone at $K = 5$, and further members mainly smooth what remains; on images 8117 and 403 the five-member average has a lower LPIPS than the twelve-member one (0.255 against 0.265 and 0.307 against 0.310). Four findings stand out:

1. *The prediction over K.* With 5,000-step members, the measured gain (mean of per-image gains, before BP) is 1.07, 1.74, 1.93, 2.16 and 2.30 dB for $K = 2, 4, 5, 8, 12$, against 1.25, 2.04, 2.22, 2.50 and 2.66 dB predicted. In the correlated version of (14), the shortfall corresponds to a pairwise error correlation ρ between 0.10 and 0.13, depending on K (0.03–0.06 for 2,000-step members), meaning 5,000-step members are not independent posterior samples. In the label-mixture model of (18) the same shortfall gives $\beta = 0.78$–0.81: if each member were the mean given its labels, about a fifth of the posterior variance would lie within labels, where no label ensemble can reach it. The fit cannot tell a true correlation from a shared bias, such as a prior whose mean is not the posterior mean: both add an error term that averaging does not remove. With 500-step members the gain is larger than predicted at every K (2.93 dB at $K = 12$). These members are not posterior samples at all. Let us denote $\mathbf{x}_k = \boldsymbol{\mu} + \mathbf{u}_k + \mathbf{v}_k$, where $\mathbf{v}_k$ is an independent optimization error with $\mathbb{E}\|\mathbf{v}_k\|^2 = a \operatorname{tr}\Sigma$. Then the gain becomes $\frac{2+a}{1+(1+a)/K} > \frac{2}{1+1/K}$, because averaging removes $\mathbf{v}_k$ as well. The sign of the difference flips between 1,000 and 2,000 steps: at 1,000 steps the gain exceeds the prediction at every K (by 0.02–0.17 dB), and at 2,000 steps it falls short at every K (by 0.07–0.15 dB).
2. *Short runs are enough for PSNR.* Twelve 500-step members (6k steps) reach 25.05 dB, level with twelve 5,000-step members (25.10 dB, 60k steps). This is 2.6 dB above the CSGM reference at 60% of its cost. However, on these six images that reference (22.45 dB), the restart chosen by LR loss, is below an average single 5,000-step member (23.16 dB). Compared to an average single member, the twelve 500-step members gain 2.65 dB over a 500-step member

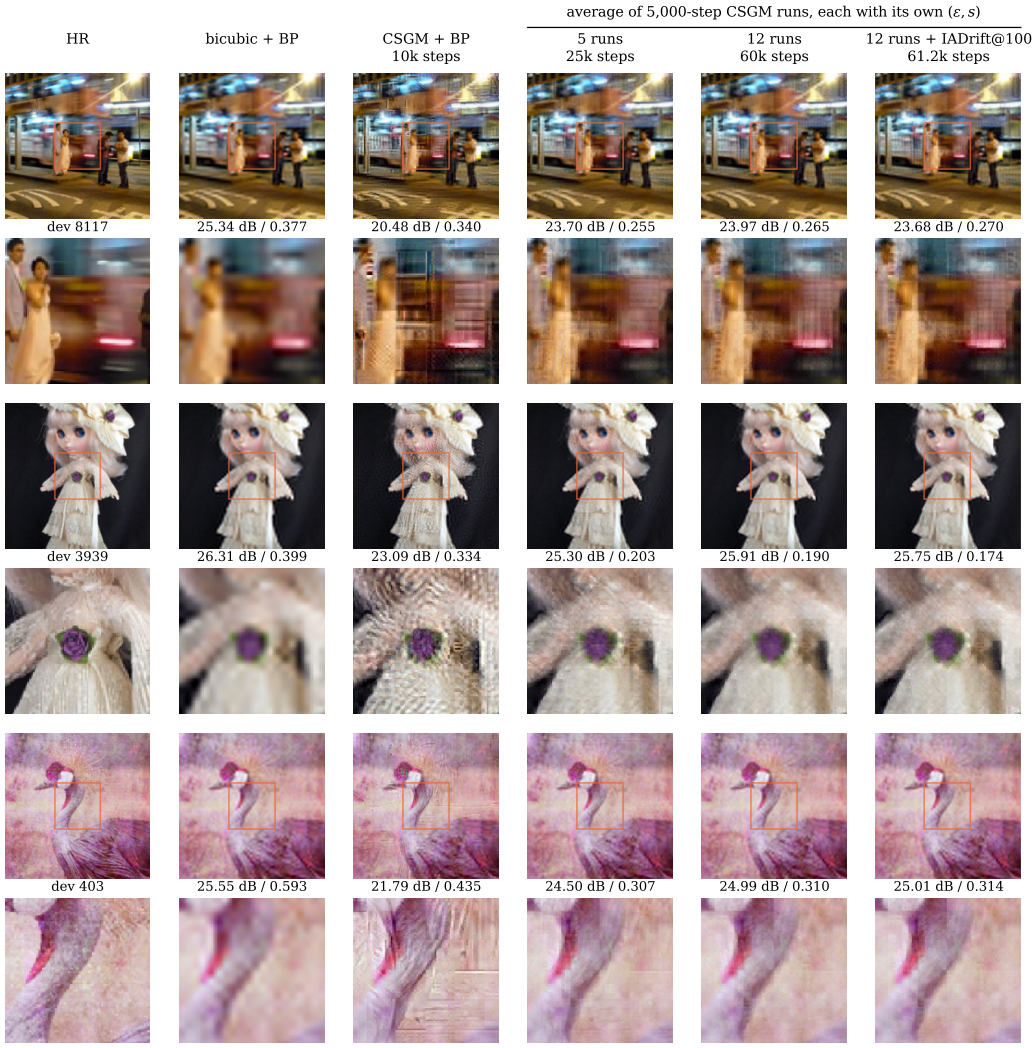


Figure 6: Development, the first three of the six images of Section 3.5 (not chosen by result), all after BP. Each image is followed by the fixed 80×80 centre crop (orange box). The last three columns are uniform averages of the first $K = 5$ or 12 CSGM runs (members $0, \ldots, K-1$, each with its own $(\boldsymbol{\varepsilon}, \mathbf{s})$); in the last column each run is first continued with IADrift@100. Below each image: PSNR / LPIPS for that image. Single images only; the panels are taken from the experiment's own figure and are drawn at about $0.8\times$ resolution.

and 1.89 dB over a 5,000-step member. LPIPS, on the other hand, still needs the long runs (0.315 against 0.246).

3. *Adaptation trades PSNR for LPIPS in an ensemble too.* Continuing every member with IADrift@100 lowers PSNR by 0.12 dB and LPIPS by 0.008 (uniform, $K = 12$; weighted: -0.19 dB and -0.010), the same tradeoff as on the test set, and gives the lowest LPIPS in the block (0.227, weighted). Averaging five IADrift@1000 runs recovers about 4 dB (19.05 to 23.00 dB) but stays below the CSGM ensembles.
4. *Grid phases or labels.* Members on shifted measurements give the highest PSNR (25.47 dB for 4 labels $\times$ 4 phases, $K = 16$) but worse LPIPS. At $K = 8$, 2 labels $\times$ 4 phases give 25.22 dB / 0.287 against 24.97 dB / 0.249 for 8 distinct labels. At the cost of ENS-U and ENS-W (25k steps; Figure 5b), five 5,000-step members with distinct labels give 24.74 dB / 0.261 uniform and 24.45 dB / 0.249 weighted, against 24.81 dB / 0.302 for ENS-U and 24.04 dB / 0.273 for ENS-W. New labels give the lower LPIPS for both averages (5 of 6 images each), but the PSNR advantage of shifts holds only for the uniform average ($+0.07$ dB, 4 of 6 images) and reverses for the weighted one (-0.41 dB, all 6 images). Overall, shifts tend to reduce distortion and new labels give better perceptual quality. One reason is the 16-pixel patch grid: members on different grid phases carry their tile seams at different offsets, so averaging them removes more of the seams. The seam score (Section 4.1) agrees with this. With uniform averages of 5,000-step members, 4 grid phases score 1.14 against 1.16 for 4 labels ($K = 4$), and 2 labels $\times$ 4 phases score 1.14 against 1.18 for 8 labels ($K = 8$; HR images 1.01). On the test set, the shift average scores 1.14 against 1.23 for the restart average (lower on 81/100 images), although the two also differ in K. This shows that shifts remove seams, not that the seams cause the PSNR difference; the Periodic Peak Score that Zhu et al. (2026) use to measure periodic artifacts would be a more direct test.

4.5 Development ablations

These are exploratory, mostly on 3 or 4 development images or on generator samples of 4 or 8 classes (Table 1(b)).

Roles of $\mathbf{s}$ and $\boldsymbol{\varepsilon}$. In samples of 8 classes, a variance decomposition attributes most of the colour (0.82) and spectrum (0.68) variation to the style labels $\mathbf{s}$; $\boldsymbol{\varepsilon}$ alone explains at most 0.10 on average (at most 0.16 for any class). Layout is shared ($\mathbf{s}$ 0.37–0.44, interaction 0.46–0.53). So the global look of the image is set by the discrete part of the input, which CSGM cannot optimize. A perturbation test points the same way: changing one of the 32 labels changes the image about 8 times more than redrawing the Gaussian noise of one 16×16 patch of $\boldsymbol{\varepsilon}$ (RMS change 0.39 against 0.046). The patch's effect is more concentrated than a label's, but it is not local: only 6% of its energy falls inside that patch's own tile and 19% inside the surrounding 3×3 tiles. Also, $\boldsymbol{\varepsilon}$ is not an image-like latent: placing the image itself in $\boldsymbol{\varepsilon}$ gives 4–11 dB, against 20–25 dB for CSGM (3 images).

Keeping $\boldsymbol{\varepsilon}$ on the Gaussian shell. The proposal expected the optimized $\boldsymbol{\varepsilon}$ to leave the Gaussian typical set, and this to be the technical core of the problem. It does leave it: after CSGM its RMS is 1.4–1.9, against 1 for a Gaussian. However, projecting it back onto the Gaussian shell does not help. Per-patch projection gains 0.13 dB after BP but worsens LPIPS (0.373 against 0.328), and projection onto the global sphere is worse on both. One might think that $\boldsymbol{\varepsilon}$ leaves the shell because it must compensate for the random labels, which set most of the image (previous paragraph). However, our data does not establish this. In the in-range experiment of Section 4.3, CSGM's $\boldsymbol{\varepsilon}$ ends as far from the true $\boldsymbol{\varepsilon}^\star$ with the true labels (relative error $\|\hat{\boldsymbol{\varepsilon}} - \boldsymbol{\varepsilon}^\star\|/\|\boldsymbol{\varepsilon}^\star\|$ of 1.68–1.97) as with random ones (1.69–2.31). So even with the right labels, CSGM finds a distant $\boldsymbol{\varepsilon}$ rather than $\boldsymbol{\varepsilon}^\star$. We did not record the RMS of $\boldsymbol{\varepsilon}$ in that experiment, so why the projection does not help remains open.

Searching the style labels. The search of Section 3.3 lowers the final LR loss from 50.5 to 30.0 on average, at $1.8\times$ the cost. After BP, PSNR changes by $+0.95$, $+0.25$, 0.00 and -0.25 dB on the four images (mean $+0.24$, median $+0.13$ dB), so one image carries most of the mean gain. LPIPS improves by 0.013 on average, but only due to one image (-0.121); it is worse on two of the four. At the screening stage, the LR loss ranks candidates almost as PSNR does (Spearman -0.85 to -0.94), but this is close to automatic, and so is the fact that on each of the four images the screened candidate with the lowest LR loss is also the one with the highest PSNR: after 500 steps a large LR loss means a poor fit of the low frequencies, which PSNR also measures. The choice between two finished runs is a harder test. On the test set, the CSGM restart with the lower LR loss has the lower HR error on 72/100 images, while on the six development images it falls below an average member (Section 4.4).

Also tried. The 16-pixel tile seams also appear in the generator's own class-conditional samples (grid score 1.08–2.11 for two samples of each of 4 classes at guidance 1.0, against 0.88–1.10 for the HR images; 1 means no grid), therefore they are a property of the generator. The inversion strengthens them: the seam score of a single CSGM member grows from 1.06 after 500 steps to 1.21 after 5,000 (6 development images), and an average of twelve 500-step members scores 1.01, the level of the HR images. Zhu et al. (2026) suggest that patch tokenization may contribute to similar periodic artifacts in SD3.5, and report that their drifting loss suppresses them. Our generator was trained with a drifting loss and still has seams, but its architecture, feature space and task differ from theirs, so this does not identify the cause. We also tried two penalties on the band above f_{LR}, on one image and untuned:

1. A *colour penalty* on high-frequency colour during CSGM cut that colour energy by about 90% (from 646 to 62) and raised PSNR after BP from 21.8 to 23.9 dB.
2. A *high-band anchor* during IADrift, which penalizes changes of the high band away from the CSGM solution it starts from, kept IADrift@1000 from collapsing (21.7 against 16.2 dB).

One image is not evidence, therefore we list this only as future work. Additionally, we ran four further ideas briefly, on a few development images, with earlier scripts which are not part of the reproduction path (kept in `scripts/dev_archive`). None of them gave a robust gain, and we do not report numbers for them:

1. *Frequency fusion* kept the reconstruction's mid band and took the other bands from bicubic; a control without any prior did better, so the gain came from BP.
2. *Soft style labels* relaxed each label to a mixture of the 64 embeddings so that gradients could optimize it, but the mixtures did not survive rounding back to discrete labels.
3. *Label swaps* changed single labels during CSGM, without a consistent improvement.
4. Starting IADrift@100 from a better label draw did not add to the gain of the label search alone.

5 Conclusion

[TODO: Conclusion.]

Discussion: why adaptation helps IAGAN but not here. IAGAN runs our task on faces: noiseless bicubic SR with BP. There, IAGAN-BP has a lower LPIPS than CSGM-BP at every scale and a higher PSNR at $\times 4$ and $\times 8$, e.g. 27.14 against 26.44 dB and LPIPS 0.092 against 0.165 on CelebA at $\times 4$; only at $\times 16$ on CelebA-HQ is its PSNR lower (25.86 against 26.20 dB) (Abu Hussein et al., 2020, Table 2). Their CSGM without BP is also far below bicubic (20.51 against 26.50 dB at $\times 4$), but with BP it is level with it (26.44 dB), whereas our CSGM + BP stays 2.82 dB below bicubic + BP. They also find IAGAN and IAGAN-BP nearly equal, which supports our reading in Section 4.2 that IADrift's gain without BP is mostly a better fit of the row space. Several differences may explain why the outcome differs (we did not test them):

1. Their generators (BEGAN, PGGAN) map a low-dimensional latent to aligned faces, a narrow domain, while ours maps image-sized noise and discrete labels to 1,000 ImageNet classes.
2. For SR they adapt for 500 (BEGAN) or 300 (PGGAN) iterations, with learning rates for $(\mathbf{z}, \boldsymbol{\theta})$ of $(10^{-4}, 10^{-4})$ (BEGAN) or $(10^{-4}, 10^{-3})$ (PGGAN), and keep the iterate with the lowest objective. Our $\eta_\theta = 10^{-5}$ is smaller, but it moves 134M transformer weights with no bottleneck.
3. Part of our CSGM's representation error comes from the random style labels (Section 4.3), which is a different kind of error from a GAN's mode collapse.
4. After BP, their CSGM is level with bicubic while ours is 2.82 dB below it, so the content our generator puts in the null space of $\mathbf{A}$ is already worse than theirs before adaptation starts.

Discussion: per-image inversion vs. the drifting objective. CSGM and IAGAN are per-measurement inverse problems, so IADrift optimizes (7) for a single image, as they do. A drifting model's prior, on the other hand, is a property of its whole pushforward $q_{\boldsymbol{\theta}} = f_{\boldsymbol{\theta}\#} p_{\boldsymbol{\varepsilon}}$: it is learned by the batch-level field $\mathbf{V}_{p,q}$ in (5), which compares many generated samples (negatives) with real images (positives). The adaptation step keeps only the measurement term and drops this distribution-matching term, meaning only the small learning rate and the limited number of steps keep $q_{\boldsymbol{\theta}}$ close to p_{data} as $\boldsymbol{\theta}$ moves. IAGAN likewise drops its adversarial training term during adaptation, so this alone does not explain why adaptation helps their GANs but not our generator. It is, however, consistent with the PSNR falling from 22.63 dB at 100 steps to 19.83 dB at 1000 steps, although we did not test it directly. Zhu et al. (2026) name the obstacle to keeping this term: drifting needs negatives generated under the same condition, while SR offers one target per input. Their answer, spatial feature drifting, treats every spatial feature vector as a sample, so one image pair yields both sets. A natural extension of IADrift is a drift regularizer during adaptation that fits $\mathbf{y}$ while the generator as a whole stays a model of natural images. Negatives are cheap in our setting, unlike theirs: the generator is stochastic, so new draws $(\boldsymbol{\varepsilon}, \mathbf{s})$ through the adapted weights $\boldsymbol{\theta}$ supply them. Positives are the hard part, because no HR image exists at test time. Spatial features of real images of class c are one candidate.

Code. The code is available at `https://github.com/zivmoshe12/IADrift`.

References

Shady Abu Hussein, Tom Tirer, and Raja Giryes. Image-adaptive GAN based reconstruction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 3121–3129, 2020. doi: 10.1609/aaai.v34i04.5708.

Mingyang Deng, He Li, Tianhong Li, Yilun Du, and Kaiming He. Generative modeling via drifting. *arXiv preprint arXiv:2602.04770*, 2026.

Wei Zhu, Kai Zhang, Yu Zheng, Zhaopeng Yang, Lei Luo, Yong Guo, and Jian Yang. DriftSR: One-step real-world image super-resolution via distribution drifting. *arXiv preprint arXiv:2610.04819*, 2026.